

Machine Learning Algorithms Laboratory

Experiment 7 Report

Experiment No.	7
Experiment Title	Dimensionality Reduction and Model Evaluation (With and Without PCA) 1
Student Name	Gautham S
Register Number	3122247001016
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

The objective of this experiment is to study the effect of dimensionality reduction using Principal Component Analysis (PCA) on the performance of several machine learning classifiers. Ten different models (SVM, Naïve Bayes, KNN, Logistic Regression, Decision Tree, Random Forest, AdaBoost, Gradient Boosting, XGBoost and Stacking) are trained and tuned both without PCA (original feature space) and with PCA (reduced feature space), and their performance is compared.

2 Theory

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique. It transforms the original correlated features into a smaller set of uncorrelated variables called principal components, which are ordered by how much variance of the original data they explain. By keeping only the top few components that explain most of the variance (for example 95%), the number of features can be reduced significantly while losing only a small amount of information. This can help reduce overfitting and training time, especially for models that are sensitive to high dimensional or correlated features, although it can also remove some useful information that a model was relying on.

3 Dataset Description

Dataset	Description
Dataset Name	Wisconsin Diagnostic Breast Cancer Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	569
Number of Features	30 numerical features
Target Classes	Benign (0, 357 samples), Malignant (1, 212 samples)

Missing Values	None
Train-Test Split	80% Train (455 samples), 20% Test (114 samples)

4 Preprocessing

The Diagnosis column ("B"/"M") was mapped to 0/1 and the ID column was dropped since it is just an identifier. There were no missing values. All 30 features were standardized using `StandardScaler`, fit only on the training data and then applied to the test data, since PCA and several of the models used here (SVM, KNN, Logistic Regression) are sensitive to feature scale.

5 PCA Analysis

PCA was first applied on the full training set (without limiting the number of components) to see how much variance each additional component explains.

```
pca_full = PCA()
pca_full.fit(X_train_scaled)

cum_variance = np.cumsum(pca_full.explained_variance_ratio_)
components_95 = np.argmax(cum_variance >= 0.95) + 1

print("Components Needed for 95% Variance:", components_95)
```

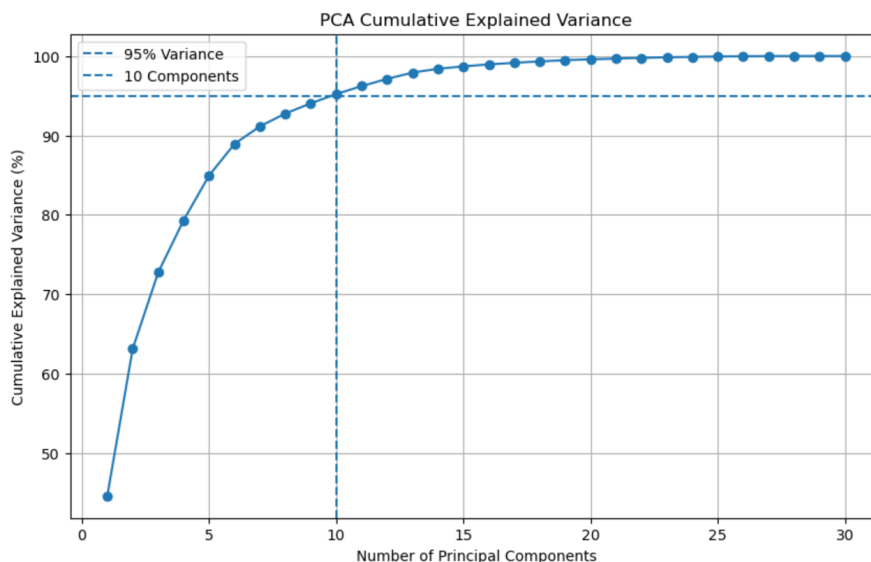


Figure 1: Cumulative Explained Variance vs Number of Components (Scree Plot)

Inference: The scree plot shows that the explained variance rises very quickly for the first few components and then flattens out. Just 10 out of the original 30 components were enough to cross the 95% explained variance mark, meaning a lot of the original 30 features carry overlapping/redundant information (which also matches the strong feature correlation seen in earlier experiments on this same dataset).

Setting	Components	Explained Variance (%)	Justification
With-PCA	10	95.21	Reduced dimensionality while retaining at least 95% of the original variance.

Table 2: PCA Configuration Summary

6 Hyperparameter Tuning

For every model, a hyperparameter grid was defined and tuned separately in the original 30-feature space (No-PCA) and in the reduced 10-component space (With-PCA) using GridSearchCV with 5-fold cross validation, scoring on F1.

```
svm_pipeline = Pipeline([("scaler", StandardScaler()), ("svm",
    SVC())])

svm_params = {
    "svm__kernel": ["linear", "rbf"],
    "svm__C": [0.1, 1, 10, 100],
    "svm__gamma": ["scale", 0.01, 0.1, 1]
}

svm_grid_no_pca = GridSearchCV(svm_pipeline, svm_params, cv=5,
    scoring="f1")
svm_grid_no_pca.fit(X_train_scaled, y_train)

svm_grid_pca = GridSearchCV(svm_pipeline, svm_params, cv=5,
    scoring="f1")
svm_grid_pca.fit(X_train_pca, y_train)
```

The same pattern (separate GridSearchCV for No-PCA and With-PCA data) was repeated for all 10 models, each with its own hyperparameter grid. The best configuration found for each model is summarized below.

Model	Best Params (No-PCA)	F1 (No-PCA)	Best Params (With-PCA)	F1 (With-PCA)
SVM	C=10, gamma=scale, kernel=rbf	0.9674	C=1, gamma=scale, kernel=linear	0.9644
Naive Bayes	var_smoothing=1e-11	0.9170	var_smoothing=1e-11	0.8824
KNN	k=3, manhattan, uniform	0.9667	k=3, euclidean, uniform	0.9605
Logistic Regression	C=1, solver=liblinear	0.9640	C=0.1, solver=liblinear	0.9701
Decision Tree	entropy, depth=5, min_split=10	0.9039	gini, depth=5, min_split=10	0.9107
Random Forest	n=200, depth=10, log2	0.9557	n=200, depth=None, sqrt	0.9430
AdaBoost	n=200, lr=0.5	0.9612	n=100, lr=1.0	0.9605
Gradient Boosting	n=200, lr=0.1, depth=2	0.9645	n=100, lr=0.2, depth=2	0.9430
XGBoost	n=50, depth=2, lr=0.2	0.9671	n=100, depth=5, lr=0.2	0.9673
Stacking	meta C=1	0.9702	meta C=1	0.9762

Table 3: Best Hyperparameters and CV F1 Score for All Models

Inference: Six out of ten models (SVM, Naive Bayes, KNN, Random Forest, AdaBoost, Gradient Boosting) actually got a slightly lower F1 score after PCA, while the other four (Logistic Regression, Decision Tree, XGBoost, Stacking) improved. Stacking was the model that benefited the most from PCA in terms of raw F1 improvement.

7 5-Fold Cross-Validation Results

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Avg
<i>No-PCA</i>						
SVM	0.9538	0.9855	0.9565	0.9706	0.9706	0.9674
Naive Bayes	0.9375	0.9714	0.8857	0.8788	0.9118	0.9170
KNN	0.9552	1.0000	0.9538	0.9394	0.9851	0.9667
Logistic Regression	0.9375	0.9565	1.0000	0.9697	0.9565	0.9640
Decision Tree	0.8657	0.9275	0.8750	0.9118	0.9394	0.9039
Random Forest	0.9552	0.9714	0.9394	0.9429	0.9697	0.9557
AdaBoost	0.9538	0.9565	0.9538	0.9565	0.9851	0.9612
Gradient Boosting	0.9552	0.9706	0.9552	0.9565	0.9851	0.9645
XGBoost	0.9697	0.9706	0.9538	0.9565	0.9851	0.9671
Stacking	0.9231	0.9714	1.0000	0.9697	0.9706	0.9670
<i>With-PCA</i>						
SVM	0.9394	0.9565	0.9714	0.9697	0.9851	0.9644
Naive Bayes	0.8710	0.9697	0.8529	0.8197	0.8986	0.8824
KNN	0.9697	0.9706	0.9375	0.9394	0.9851	0.9605
Logistic Regression	0.9697	0.9565	0.9697	0.9697	0.9851	0.9701
Decision Tree	0.8955	0.9565	0.9118	0.8485	0.9412	0.9107
Random Forest	0.9538	0.9714	0.9412	0.9091	0.9394	0.9430
AdaBoost	0.9206	0.9851	0.9706	0.9412	0.9851	0.9605
Gradient Boosting	0.9231	0.9577	0.9412	0.9394	0.9538	0.9430
XGBoost	0.9538	0.9714	0.9565	0.9851	0.9697	0.9673
Stacking	0.9697	0.9565	1.0000	0.9697	0.9851	0.9762

Table 4: 5-Fold Cross-Validation F1 Scores (No-PCA vs With-PCA)

Model	Avg (No-PCA)	Avg (With-PCA)	Change	Effect
SVM	0.9674	0.9644	-0.0030	Decreased
Naive Bayes	0.9170	0.8824	-0.0346	Decreased
KNN	0.9667	0.9605	-0.0062	Decreased
Logistic Regression	0.9640	0.9701	+0.0061	Improved
Decision Tree	0.9039	0.9107	+0.0068	Improved
Random Forest	0.9557	0.9430	-0.0127	Decreased
AdaBoost	0.9612	0.9605	-0.0007	Decreased
Gradient Boosting	0.9645	0.9430	-0.0215	Decreased
XGBoost	0.9671	0.9673	+0.0002	Improved
Stacking	0.9670	0.9762	+0.0092	Improved

Table 5: Effect of PCA on Average Cross-Validation F1 Score

Inference: Naive Bayes was hurt the most by PCA (F1 dropped by 0.0346), which makes sense since it assumes feature independence, and PCA components are linear combinations of the original correlated features which changes their distribution and can violate this assumption more. Ensemble tree-based models like Random Forest and Gradient Boosting also dropped a bit, probably because reducing to 10 components takes away some of the specific original features that the trees could split on cleanly. Stacking and Logistic Regression, on the other hand, actually improved with PCA, possibly because removing some of the noise/redundant correlated features helped the linear meta-learner and base models generalize a bit better.

8 Test Set Performance

Model	Accuracy (No-PCA)	F1 (No-PCA)	Accuracy (With-PCA)	F1 (With-PCA)
SVM	0.9737	0.9630	0.9649	0.9512
Naive Bayes	0.9211	0.8889	0.8947	0.8571
KNN	0.9649	0.9500	0.9474	0.9250
Logistic Regression	0.9737	0.9639	0.9825	0.9756
Decision Tree	0.9561	0.9383	0.9386	0.9136
Random Forest	0.9561	0.9367	0.9474	0.9268
AdaBoost	0.9737	0.9630	0.9649	0.9512
Gradient Boosting	0.9649	0.9500	0.9561	0.9383
XGBoost	0.9649	0.9500	0.9737	0.9639
Stacking	0.9912	0.9880	0.9825	0.9756

Table 6: Test Set Accuracy and F1 Score (No-PCA vs With-PCA)

Inference: Stacking without PCA gave the single best test set result overall, with an accuracy of 99.12% and F1 of 0.988. Among the With-PCA models, Logistic Regression was the best (98.25% accuracy). This matches the cross-validation results reasonably well, both Stacking and Logistic Regression were among the models that improved or stayed strong, confirming these results are not just a one-off on the test split.

9 ROC and Precision-Recall Curves

Based on the test set results, three of the strongest models, Stacking (No-PCA), Logistic Regression (With-PCA) and XGBoost (With-PCA), were selected for a closer look using ROC and Precision-Recall curves.

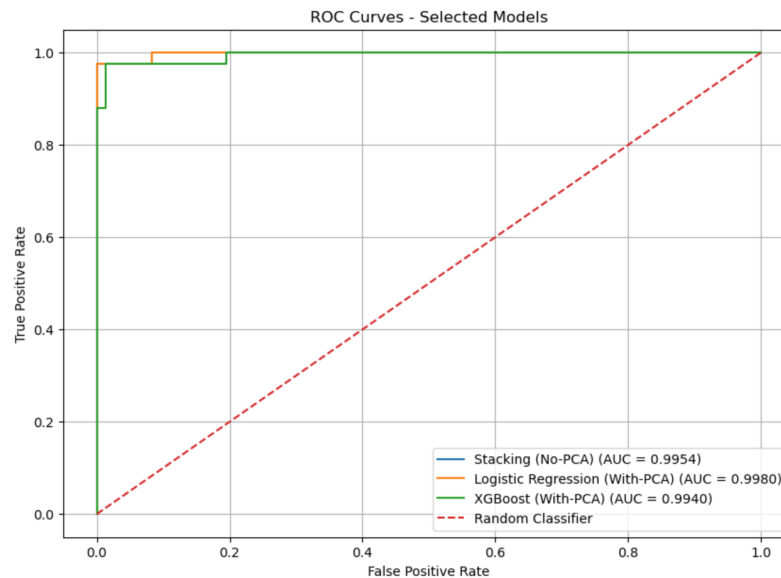


Figure 2: ROC Curves for Selected Top Models

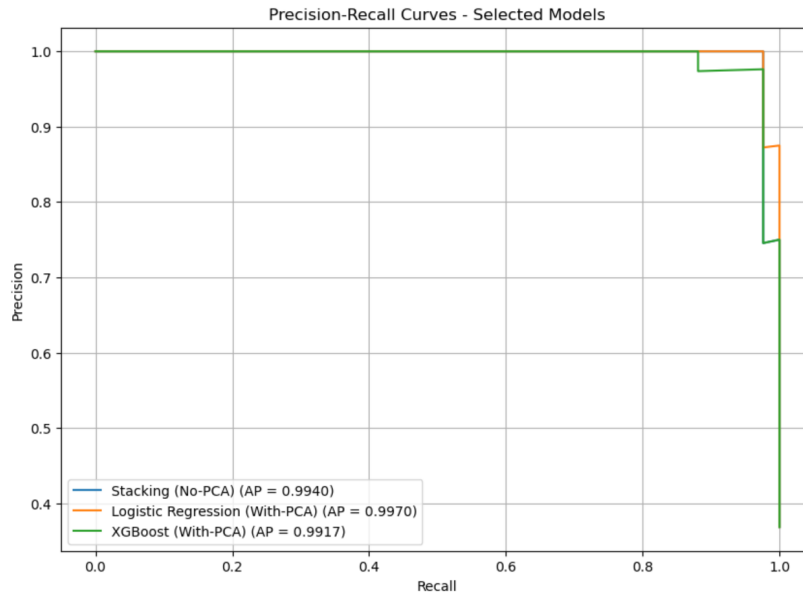


Figure 3: Precision-Recall Curves for Selected Top Models

Model	ROC-AUC	Average Precision (AP)
Stacking (No-PCA)	0.9954	0.9940
Logistic Regression (With-PCA)	0.9980	0.9970
XGBoost (With-PCA)	0.9940	0.9917

Table 7: ROC-AUC and Average Precision of Selected Models

Inference: All three selected models have a ROC-AUC above 0.99, meaning they are all excellent at separating malignant and benign cases across thresholds. Logistic Regression (With-PCA) actually has the highest ROC-AUC (0.998) among the three, even though Stacking (No-PCA) had the best accuracy/F1 at the default 0.5 threshold. This again shows that a model can be a very slightly better overall ranker (higher AUC) while another model wins at one specific threshold.

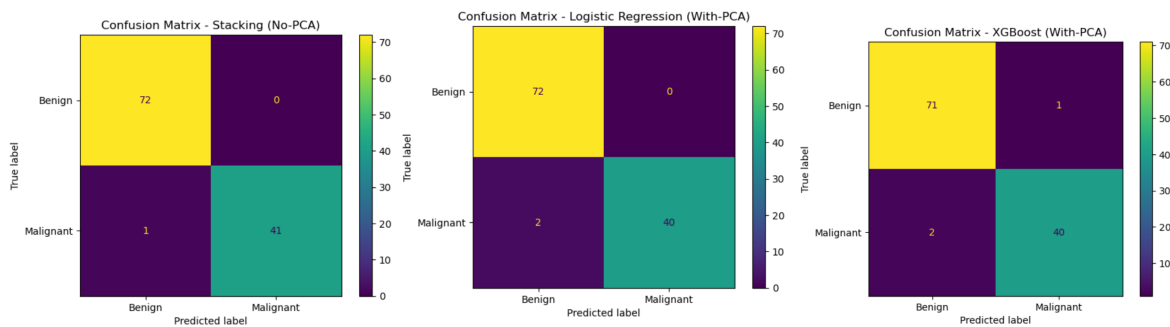


Figure 4: Confusion Matrices - Stacking (No-PCA), Logistic Regression (With-PCA), XGBoost (With-PCA)

Inference: Stacking (No-PCA) makes the fewest mistakes overall on the test set among the three, matching its top accuracy. Logistic Regression (With-PCA) and XGBoost (With-PCA) both make only a couple of misclassifications each, and none of the

three models have a large number of false negatives, which is reassuring since missing a malignant case is the more serious kind of error in this context.

10 Observation Questions

1. Which models improved most with PCA? Which did not? Why?

Stacking improved the most in cross-validation F1 (+0.0092), followed by Decision Tree (+0.0068) and Logistic Regression (+0.0061). Naive Bayes was hurt the most (-0.0346), followed by Gradient Boosting (-0.0215) and Random Forest (-0.0127). Models that rely on clean, individually meaningful features (Naive Bayes's independence assumption, tree-based splits) tend to lose useful information when features are compressed into PCA components, while linear/meta models that combine many features together (Logistic Regression, the Stacking meta-learner) seem to benefit slightly from the noise reduction PCA provides.

2. Did PCA reduce variance across folds (more stable results)?

Looking at the fold-wise table, the results are mixed rather than a clean story either way. For some models like Naive Bayes, the fold values with PCA actually swing more widely (0.82 to 0.97) than without PCA (0.88 to 0.97), so PCA did not make it more stable. For Logistic Regression and Stacking, the fold-to-fold spread with PCA looks similar to or slightly tighter than without PCA. So in this experiment, PCA did not consistently reduce fold-to-fold variance across all models, it depended on the specific model.

3. For high-dimensional data, was PCA beneficial in reducing overfitting?

This dataset only has 30 features and 455 training samples, so it is not extremely high-dimensional to begin with, and most models here were not showing severe overfitting even without PCA. Because of this, PCA's benefit for reducing overfitting was limited in this particular experiment, its effect was more about changing which model performs best rather than clearly reducing overfitting for every model. On genuinely higher-dimensional data (hundreds or thousands of features), PCA would likely have a bigger impact on both overfitting and training time.

4. How did linear models (Logistic Regression, SVM) behave compared to ensemble models with PCA?

Logistic Regression improved with PCA while SVM slightly decreased, so even between the two linear/margin-based models the effect was not identical. Among the ensemble models, XGBoost and Stacking (which uses a linear meta-learner) improved slightly, while Random Forest, AdaBoost and Gradient Boosting all decreased somewhat. So overall, models that either directly are linear or use a linear layer at the end (Logistic Regression, Stacking) tended to benefit a bit more from PCA than tree-based ensembles that rely on splitting individual original features.

5. Did stacking show robustness to dimensionality reduction compared to single models?

Yes, Stacking performed well both with and without PCA (0.9670 and 0.9762 average CV F1 respectively, and 0.9912/0.9825 test accuracy), staying among the top 2-3 models in both settings. This robustness likely comes from combining multiple different base models (SVM, Naive Bayes, Decision Tree), so even if PCA makes the input slightly less favorable for one of the base learners, the others and the meta-learner can still compensate, unlike a single model which has no such backup.

11 Conclusion

In this experiment, ten different classifiers were evaluated on the Wisconsin Breast Cancer dataset both with and without PCA. Stacking (No-PCA) gave the best overall test accuracy (99.12%) and F1 score (0.988), while Logistic Regression (With-PCA) was the best performing model in the reduced feature space and also had the highest ROC-AUC (0.998) overall. PCA improved 4 out of 10 models (Logistic Regression, Decision Tree, XGBoost, Stacking) and slightly hurt the remaining 6, showing that dimensionality reduction is not universally beneficial, its effect depends heavily on how a specific model uses its input features. Simpler linear models and stacked ensembles with a linear meta-learner tended to benefit slightly from PCA, while models like Naive Bayes and tree-based ensembles that rely on interpreting individual original features performed a bit better without it. Overall, whether to use PCA should be decided on a per-model basis using cross-validation, rather than assumed to always help.

12 References

1. Scikit-learn Documentation: PCA. <https://scikit-learn.org/stable/modules/decomposition.html#pca>
2. Scikit-learn Documentation: Ensemble Methods. <https://scikit-learn.org/stable/modules/ensemble.html>
3. XGBoost Documentation. <https://xgboost.readthedocs.io/>
4. Wisconsin Diagnostic Breast Cancer Dataset, UCI Machine Learning Repository.